MicroserviceBase

v. 0.1.1

Nguyen Huynh Tri Cuong

02.02.2023

Contents

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceBase**.

The sources of **MicroserviceBase** are available in GitHub.

Informations about how to install the **MicroserviceBase** can be found in the README.

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices via RabbitMQ.

3.1 Function: parse_args

3.2 Function: main

Chapter 4

start_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

4.1 Function: parse_args

4.2 Function: main

Chapter 5

start_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

5.1 Function: parse_args

5.2 Function: main

Chapter 6

__init__.py

biplist -- a library for reading and writing binary property list files.

Binary Property List (plist) files provide a faster and smaller serialization format for property lists on OS X. This is a library for generating binary plists which can be read by OS X, iOS, or other clients.

The API models the plistlib API, and will call through to plistlib when XML serialization or deserialization is required.

To generate plists with UID values, wrap the values with the Uid object. The value must be an int.

To generate plists with NSData/CFData values, wrap the values with the Data object. The value must be a string.

Date values can only be datetime.datetime objects.

The exceptions InvalidPlistException and NotBinaryPlistException may be thrown to indicate that the data cannot be serialized or deserialized as a binary plist.

Plist generation example:

> from biplist import * from datetime import datetime plist = {'aKey':'aValue', '0':1.322, 'now':datetime.now(), 'list':[1,2,3], 'tuple':('a','b','c') } try: writePlist(plist, "example.plist") except (InvalidPlistException, NotBinaryPlistException), e: print "Something bad happened:", e

Plist parsing example:

> from biplist import * try: plist = readPlist("example.plist") print plist except (InvalidPlistException, NotBinaryPlistException), e: print "Not a plist:", e

6.1 Function: readPlist

Raises NotBinaryPlistException, InvalidPlistException microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---wrapdataobject ==========

6.2 Function: writePlist

6.3 Function: readPlistFromString

6.4 Function: writePlistToString

6.5 Function: is_stream_binary_plist

6.6 Class: Uid

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Uid
```

Wrapper around integers for representing UID values. This is used in keyed archiving. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---data ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Data
```

Wrapper around bytes to distinguish Data values. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---invalidplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import InvalidPlistException
```

Raised when the plist is incorrectly formatted. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---notbinaryplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import NotBinaryPlistException
```

Raised when a binary plist was expected but not encountered. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistreader ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistReader
```

6.6.1 Method: parse

6.6.2 Method: reset

6.6.3 Method: readRoot

6.6.4 Method: setCurrentOffsetToObjectNumber

6.6.5 Method: beginOffsetProtection

6.6.6 Method: endOffsetProtection

6.6.7 Method: readObject

6.6.8 Method: readContents

6.6.9 Method: readInteger

6.6.10 Method: readReal

6.6.11 Method: readRefs

6.6.12 Method: readArray

6.6.13 Method: readDict

6.6.14 Method: readAsciiString

6.6.15 Method: readUnicode

6.6.16 Method: readDate

6.6.17 Method: readData

6.6.18 Method: readUid

6.6.19 Method: getSizedInteger

Numbers of 8 bytes are signed integers when they refer to numbers, but unsigned otherwise. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---hashablewrapper ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import HashableWrapper
```

6.7 Class: BoolWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import BoolWrapper
```

6.8 Class: FloatWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import FloatWrapper
```

6.9 Class: StringWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import StringWrapper
```

6.9.1 Method: encodingMarker

6.10 Class: PlistWriter

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistWriter
```

6.10.1 Method: reset

6.10.2 Method: positionOfObjectReference

If the given object has been written already, return its position in the offset table. Otherwise, return None. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeroot ----------

Strategy is: - write header - wrap root object so everything is hashable - compute size of objects which will be written - need to do this in order to know how large the object refs will be in the list/dict/set reference lists - write objects - keep objects in writtenReferences - keep positions of object references in referencePositions - write object references with the length computed previously - computer object reference length - write object reference positions - write trailer microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-beginrecursionprotection ----------

6.10.3 Method: endRecursionProtection

6.10.4 Method: wrapRoot

6.10.5 Method: incrementByteCount

6.10.6 Method: computeOffsets

6.10.7 Method: writeObjectReference

Tries to write an object reference, adding it to the references table. Does not write the actual object bytes or set the reference position. Returns a tuple of whether the object was a new reference (True if it was, False if it already was in the reference table) and the new output. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeobject ----------

Serializes the given object to the output. Returns output. If setReferencePosition is True, will set the position the object was written. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeoffsettable ----------

Writes all of the object reference offsets. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-binaryreal ----------

6.10.8 Method: binaryInt

6.10.9 Method: intSize

Returns the number of bytes necessary to store the given integer. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-realsize ----------

Chapter 7

badge.py

7.1 Function: badge_disk_icon

Chapter 8

colors.py

8.1 Function: isAColor

8.2 Function: parseColor

8.3 Class: Color

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Color
```

8.3.1 Method: to_rgb

8.4 Class: RGB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import RGB
```

8.4.1 Method: to_rgb

8.5 Class: HSL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HSL
```

8.5.1 Method: to_rgb

8.6 Class: HWB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HWB
```

8.6.1 Method: to_rgb

8.7 Class: CMYK

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import CMYK
```

8.7.1 Method: to_rgb

8.8 Class: Gray

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Gray
```

8.8.1 Method: to_rgb

8.9 Class: ColorParser

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import ColorParser
```

8.9.1 Method: skipws

8.9.2 Method: expect

8.9.3 Method: expectEnd

8.9.4 Method: getToken

8.9.5 Method: parseNumber

8.9.6 Method: parseColor

8.9.7 Method: parseRGB

8.9.8 Method: parseHSL

8.9.9 Method: parseHWB

8.9.10 Method: parseCMYK

8.9.11 Method: parseGray

8.9.12 Method: parseValue

8.9.13 Method: parseAngle

Chapter 9

core.py

9.1 Function: build_dmg

9.2 Class: DMGError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.core
↪  import DMGError
```

Chapter 10

buddy.py

10.1 Class: BuddyError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import BuddyError
```

10.2 Class: Block

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Block
```

10.2.1 Method: close

10.2.2 Method: flush

10.2.3 Method: invalidate

10.2.4 Method: zero_fill

10.2.5 Method: tell

10.2.6 Method: seek

10.2.7 Method: read

10.2.8 Method: write

10.2.9 Method: insert

10.2.10 Method: delete

10.3 Class: Allocator

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Allocator
```

10.3.1 Method: open

10.3.2 Method: close

10.3.3 Method: flush

10.3.4 Method: read

Read data at ‘offset', or raise an exception. ‘size_or_format' may either be a byte count, in which case we return raw data, or a format string for ‘struct.unpack', in which case we work out the size and unpack the data before returning it. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-write ----------

Write data at ‘offset', or raise an exception. ‘data_or_format' may either be the data to write, or a format string for ‘struct.pack', in which case we pack the additional arguments and write the resulting data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-get-block ----------

10.3.5 Method: allocate

Allocate or reallocate a block such that it has space for at least ‘bytes' bytes. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-release ----------

10.3.6 Method: iterkeys

10.3.7 Method: keys

Chapter 11

store.py

11.1 Class: ILocCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import ILocCodec
```

11.1.1 Method: encode

11.1.2 Method: decode

11.2 Class: PlistCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import PlistCodec
```

11.2.1 Method: encode

11.2.2 Method: decode

11.3 Class: BookmarkCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import BookmarkCodec
```

11.3.1 Method: encode

11.3.2 Method: decode

11.4 Class: DSStoreEntry

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
 ↪  import DSStoreEntry
```

Holds the data from an entry in a `.DS_Store` file. Note that this is not meant to represent the entry itself---i.e. if you change the type or value, your changes will *not* be reflected in the underlying file.

If you want to make a change, you should either use the `DSStore` object's `DSStore.insert` method (which will replace a key if it already exists), or the mapping access mode for `DSStore` (often simpler anyway). microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-read ----------

Read a `.DS_Store` entry from the containing Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-byte-length ----------

Compute the length of this entry, in bytes microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-write ----------

Write this entry to the specified Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore ==========

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
 ↪  import DSStore
```

Python interface to a `.DS_Store` file. Works by manipulating the file on the disk---so this code will work with `.DS_Store` files for *very* large directories.

A `DSStore` object can be used as if it was a mapping, e.g.:

```
d['foobar.dat']['Iloc']
```

will fetch the "Iloc" record for "foobar.dat", or raise `KeyError` if there is no such record. If used in this manner, the `DSStore` object will return (type, value) tuples, unless the type is "blob" and the module knows how to decode it.

Currently, we know how to decode "Iloc", "bwsp", "lsvp", "lsvP" and "icvp" blobs. "Iloc" decodes to an (x, y) tuple, while the others are all decoded using `biplist`.

Assignment also works, e.g.:

```
d['foobar.dat']['note'] = ('ustr', u'Hello World!')
```

as does deletion with `del`:

```
del d['foobar.dat']['note']
```

This is usually going to be the most convenient interface, though occasionally (for instance when creating a new `.DS_Store` file) you may wish to drop down to using `DSStoreEntry` objects directly. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-open ----------

Open a `.DS_Store` file; pass either a Python file object, or a filename in the `file_or_name` argument and a file access mode in the `mode` argument. If you are creating a new file using the "w" or "w+" modes, you may also specify a list of entries with which to initialise the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-flush ----------

Flush any dirty data back to the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-close ----------

Flush dirty data and close the underlying file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-insert ----------

Insert `entry` (which should be a `DSStoreEntry`) into the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-delete ----------

Delete an item, identified by `filename` and `code` from the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-find ----------

Returns a generator that will iterate over matching entries in the B-Tree.

Chapter 12

alias.py

12.1 Function: encode_utf8

12.2 Function: decode_utf8

12.3 Class: AppleShareInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import AppleShareInfo
```

12.4 Class: VolumeInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import VolumeInfo
```

12.4.1 Method: filesystem_type

12.5 Class: TargetInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import TargetInfo
```

12.6 Class: Alias

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import Alias
```

12.6.1 Method: from_bytes

Construct an `Alias` object given binary Alias data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-for-file ----------

Create an `Alias` that points at the specified file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-to-bytes ----------

Returns the binary representation for this `Alias`.

Chapter 13

bookmark.py

13.1 Function: iteritems

13.2 Class: Data

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Data
```

13.3 Class: URL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import URL
```

13.3.1 Method: absolute

Return an absolute URL. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Bookmark
```

13.3.2 Method: from_bytes

Create a `Bookmark` given byte data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-get ----------

Lookup the value for a given key, returning a default if not present. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-to-bytes ----------

Convert this `Bookmark` to a byte representation. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-for-file ----------

Construct a `Bookmark` for a given file.

Chapter 14

osx.py

14.1 Function: getattrlist

14.2 Function: fgetattrlist

14.3 Function: statfs

14.4 Function: fstatfs

14.5 Class: attrlist

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrlist
```

14.6 Class: attribute_set_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attribute_set_t
```

14.7 Class: fsobj_id_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsobj_id_t
```

14.8 Class: timespec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import timespec
```

14.9 Class: attrreference_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrreference_t
```

14.10 Class: fsid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsid_t
```

14.11 Class: guid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import guid_t
```

14.12 Class: kauth_ace

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_ace
```

14.13 Class: kauth_acl

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_acl
```

14.14 Class: kauth_filesec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_filesec
```

14.15 Class: diskextent

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import diskextent
```

14.16 Class: Point

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import Point
```

14.17 Class: Rect

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import Rect
```

14.18 Class: FileInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FileInfo
```

14.19 Class: FolderInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FolderInfo
```

14.20 Class: FinderInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FinderInfo
```

14.21 Class: vol_capabilities_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_capabilities_attr_t
```

14.22 Class: vol_attributes_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_attributes_attr_t
```

14.23 Class: struct_statfs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import struct_statfs
```

Chapter 15

utils.py

15.1 Class: UTC

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.utils
↪  import UTC
```

15.1.1 Method: utcoffset

15.1.2 Method: dst

15.1.3 Method: tzname

Chapter 16

launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices via RabbitMQ.

16.1 Function: parse_args

16.2 Function: main

Chapter 17

ClewareAccessHelper.py

17.1 Class: ClewareAccessHelper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths ----------

Import all modules from given paths.

Arguments:

- `paths`
  / *Condition*: required / *Type*: list /
  List of paths to import modules from.

17.1.1 Method: load_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config ----------

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config ----------

Save Cleware ports' config to file. Args: config_json: data to be saved.

Returns: res: saved result.

17.1.2 Method: init_cleware

17.1.3 Method: open_cleware

17.1.4 Method: close_cleware

17.1.5 Method: get_handle

17.1.6 Method: set_value

17.1.7 Method: get_value

17.1.8 Method: set_switch

17.1.9 Method: set_switch_by_port_name

17.1.10 Method: get_switch

17.1.11 Method: get_version

17.1.12 Method: get_usb_type

17.1.13 Method: get_serial_number

17.1.14 Method: valid_ser_number

17.1.15 Method: get_hw_version

17.1.16 Method: is_ampel

17.1.17 Method: iox

17.1.18 Method: get_all_devices_state

Chapter 18

ClewareAccessHelperAbs.py

18.1 Class: ClewareAccessHelperAbs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware
---

18.1.1 Method: open_cleware

18.1.2 Method: close_cleware

18.1.3 Method: set_value

18.1.4 Method: get_value

18.1.5 Method: set_switch

18.1.6 Method: get_switch

18.1.7 Method: get_handle

18.1.8 Method: get_version

18.1.9 Method: get_usb_type

18.1.10 Method: get_usb_type

18.1.11 Method: get_serial_number

18.1.12 Method: valid_ser_number

18.1.13 Method: get_hw_version

18.1.14 Method: is_ampel

18.1.15 Method: iox

Chapter 19

ClewareAccessHelperLinux.py

19.1 Class: ClewareAccessHelperLinux

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

19.1.1 Method: init_cleware

19.1.2 Method: open_cleware

19.1.3 Method: close_cleware

19.1.4 Method: get_handle

19.1.5 Method: set_value

19.1.6 Method: get_value

19.1.7 Method: set_switch

19.1.8 Method: get_switch

19.1.9 Method: get_version

19.1.10 Method: get_usb_type

19.1.11 Method: get_serial_number

19.1.12 Method: valid_ser_number

19.1.13 Method: get_hw_version

19.1.14 Method: is_ampel

19.1.15 Method: iox

19.1.16 Method: get_all_sw_state

Chapter 20

ClewareAccessHelperWindows.py

20.1 Class: ClewareAccessHelperWindows

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

20.1.1 Method: init_cleware

20.1.2 Method: open_cleware

20.1.3 Method: close_cleware

20.1.4 Method: get_handle

20.1.5 Method: set_value

20.1.6 Method: get_value

20.1.7 Method: set_switch

20.1.8 Method: get_switch

20.1.9 Method: get_version

20.1.10 Method: get_usb_type

20.1.11 Method: get_serial_number

20.1.12 Method: valid_ser_number

20.1.13 Method: get_hw_version

20.1.14 Method: is_ampel

20.1.15 Method: iox

Chapter 21

ServiceCleware.py

21.1 Class: ServiceCleware

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceCleware
↪  import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

21.1.1 Method: svc_api_get_all_devices_state

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

21.1.2 Method: svc_api_set_switch

Set state for a Cleware device's switch.

Arguments:

- `device_no`
  / *Condition*: required / *Type*: str /
  Cleware device's number.
- `switch_id`
  / *Condition*: required / *Type*: str /
  Switch number to turn on or off.
- `state`
  / *Condition*: required / *Type*: str /
  State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

21.1.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(*no returns*)

Chapter 22

ServiceLogger.py

22.1 Class: ServiceLogger

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceLogger
↪  import ServiceLogger
```

Logger class. microservicebase-microservicemanagergui-python-services-clewareservice-servicelogger-servicelogger-set-logger ----------

22.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

22.1.2 Method: log

22.1.3 Method: log_info

22.1.4 Method: log_debug

22.1.5 Method: log_warning

22.1.6 Method: log_error

22.1.7 Method: log_critical

Chapter 23

Utils.py

23.1 Class: Singleton

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

23.2 Class: Utils

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-python-services-clewareservice-utils-utils-get-all-descendant-classes ----------

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

23.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

23.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

23.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

23.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

23.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

23.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

23.3 Class: Job

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Job
```

23.3.1 Method: stop

23.3.2 Method: run

Chapter 24

__main__.py

24.1 Function: main

24.2 Function: signal_handler

Chapter 25

main.py

25.1 Function: main

25.2 Function: signal_handler

Chapter 26

start_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

26.1 Function: parse_args

26.2 Function: main

Chapter 27

start_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

27.1 Function: parse_args

27.2 Function: main

Chapter 28

eventbus_config.py

28.1 Class: EventBusConfig

Imported by:

```
from MicroserviceBase.adapters.config.eventbus_config import EventBusConfig
```

Configuration for EventBusClient-based adapters.

Wraps a JSONP config file path used by EventBusClient.from_config_sync().

28.1.1 Method: load_config

Load and return the parsed config as a dict.

Requires EventBusClient to be installed.

Returns:

/ *Type*: dict /

Parsed configuration dictionary.

28.1.2 Method: to_config_source

Load config and return as a dict with optional field overrides.

Useful for creating derivative clients (e.g., registry or fanout) that share connection settings but differ in exchange configuration.

Arguments:

- `overrides`

  / *Condition*: optional / *Type*: keyword arguments /

  Fields to override in the loaded config (e.g., exchange_name, exchange_handler).

Returns:

/ *Type*: dict /

Config dictionary with overrides applied.

Chapter 29

rabbitmq_config.py

29.1 Class: RabbitMQConfig

Imported by:

```
from MicroserviceBase.adapters.config.rabbitmq_config import RabbitMQConfig
```

Configuration for connecting to RabbitMQ.

29.1.1 Method: to_connection_params

Convert to pika ConnectionParameters kwargs.

Returns:

dict suitable for pika.ConnectionParameters(**kwargs).

29.1.2 Method: from_cmd_args

Parse RabbitMQ config from command-line arguments and environment variables.

Arguments:

- `cmd_args`
  / *Condition*: optional / *Type*: list /
  List of CLI arguments, or None for sys.argv.
- `service_name`
  / *Condition*: optional / *Type*: str /
  Name of the service (used in help text).

Returns:

RabbitMQConfig instance.

Chapter 30

local_hub_manager.py

Local Hub Manager — manages a local ProcessHub instance lifecycle.

Wraps ProcessHub APIs (lazy import) so the MicroserviceManager GUI can start/stop a ProcessHub on the local machine, manage processes, and optionally join a fleet as an agent.

30.1 Class: LocalHubManager

Imported by:

```
from MicroserviceBase.adapters.local_hub.local_hub_manager import LocalHubManager
```

Manages a local ProcessHub instance lifecycle. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-start-hub ----------

Start a local ProcessHub server.

Args: mode: "standalone" or "agent" (join fleet). xpub_port: ZMQ XPUB port for the broker. xsub_port: ZMQ XSUB port for the broker. orchestrator_url: Fleet orchestrator ZMQ address (agent mode). hub_id: Hub identifier (agent mode). hub_name: Human-readable hub name (agent mode). process_config: Dict of {name: config} for processes.

Returns: Status dict.

30.1.1 Method: stop_hub

Stop the local hub. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-get-status ----------

Get current hub status. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-start-processes ----------

Start named processes. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-stop-processes ----------

Stop named processes. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-get-config ----------

Get all process configurations. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-add-config ----------

Add a new process configuration. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-update-config ----------

Update an existing process configuration. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-remove-config ----------

Remove a process configuration. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-remove-service ----------

Remove a service — delete config and managed service folder.

Only deletes the folder if it lives inside the managed <config_dir>/services/ directory (never touches external paths). Stops the process first if it is still running. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-get-service-log ----------

Read the last *tail* lines of a process log file. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-get-services-dir ----------

Return absolute path to <config_dir>/services/, creating it if needed. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-import-service ----------

Import a microservice from a folder or base64-encoded ZIP.

Validates structure, copies files into <config_dir>/services/{name}/, and creates a hub config entry using the ${python} placeholder.

Returns {"success": bool, "message": str, "warnings": [...]}. microservicebase-adapters-local-hub-local-hub-manager-localhubmanager-reset ----------

Reset the hub (stop processes, clear connections).

Chapter 31

service_executor.py

Service-aware process executor.

Wraps a `SimpleExecutor` and attempts graceful RPC shutdown (`svc_api_shutdown`) before falling back to signal-based stop.

31.1 Class: ServiceExecutor

Imported by:

```
from MicroserviceBase.adapters.local_hub.service_executor import ServiceExecutor
```

Process executor that sends an RPC shutdown to MicroserviceBase services.

Delegates all operations to a wrapped `SimpleExecutor`. On `stop()`, it first tries to send `svc_api_shutdown` via RabbitMQ to the service's queue. If the service exits within `shutdown_timeout` seconds the process is considered stopped. Otherwise it falls back to the delegate's signal-based stop (CTRL_BREAK_EVENT on Windows, SIGTERM on Linux). microservicebase-adapters-local-hub-service-executor-serviceexecutor-start ----------

31.1.1 Method: get_log_path

Public accessor for log file path. microservicebase-adapters-local-hub-service-executor-serviceexecutor-read-log ----------

Public method to read process log. Returns the text content. microservicebase-adapters-local-hub-service-executor-serviceexecutor-is-running ----------

31.1.2 Method: get_pid

31.1.3 Method: stop

Chapter 32

amqp_registry_adapter.py

32.1 Class: AMQPRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.amqp_registry_adapter import
↪  AMQPRegistryAdapter
```

AMQP implementation of the ServiceRegistryPort interface.
Uses RabbitMQ topic exchange for service registration events and fanout exchange for realtime update broadcasts.

32.1.1 Method: register

Register a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

32.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

32.1.3 Method: subscribe_to_events

Subscribe to service registration events.
Runs in the calling thread (blocking). Typically called from a daemon thread.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

32.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

32.1.5 Method: subscribe_to_updates

Subscribe to the realtime update fanout exchange.

Unlike `subscribe_to_events` (which listens for individual on/off events), this subscribes to the full services dict broadcast by the Registry after each change. Uses an exclusive temporary queue to avoid competing consumers.

Runs in the calling thread (blocking). Typically called from a daemon thread.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(services_info_dict) called with the full services dict.

32.1.6 Method: get_update_channel_name

Return the realtime update exchange name.

32.1.7 Method: cleanup_update_exchange

Delete the realtime update exchange (for cleanup on shutdown).

32.1.8 Method: cleanup

Close event subscription resources and clean up the update exchange.

Chapter 33

eventbus_registry_adapter.py

33.1 Class: EventBusRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.eventbus_registry_adapter import
↪  EventBusRegistryAdapter
```

EventBusClient implementation of the ServiceRegistryPort interface.

Uses EventBusClient with TopicExchangeHandler for service registration events and a FanoutExchangeHandler for realtime update broadcasts.

Both clients are created from the same JSONP config with exchange overrides via `EventBusConfig.to_config_source()`.

33.1.1 Method: register

Register a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

33.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

33.1.3 Method: subscribe_to_events

Subscribe to service registration events.

Blocks the calling thread. Typically called from a daemon thread. The handler receives the pika-style callback signature (ch, method, properties, body) for backward compatibility.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

33.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

33.1.5 Method: get_update_channel_name

Return the realtime update exchange name.

33.1.6 Method: cleanup

Close all EventBusClient connections.

Chapter 34

eventbus_adapter.py

34.1 Class: _ChannelProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _ChannelProxy
```

Lightweight proxy that mimics a pika channel for the domain handler.
Translates basic_publish and basic_ack calls into EventBusClient operations.

34.1.1 Method: basic_publish

Publish a reply message via the EventBusClient.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (ignored, EventBusClient uses its configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the reply (the reply_to value).
- `properties`
  / *Condition*: optional / *Type*: object /
  Properties object with correlation_id attribute.
- `body`
  / *Condition*: optional / *Type*: str or bytes /
  The message body (JSON string or bytes).

34.1.2 Method: basic_ack

Acknowledge a message (no-op for EventBusClient which auto-acks).

34.2 Class: _MethodProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _MethodProxy
```

Lightweight proxy that mimics a pika method frame.

34.3 Class: _PropertiesProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _PropertiesProxy
```

Lightweight proxy that mimics pika BasicProperties.

34.4 Class: EventBusTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import
↪  EventBusTransportAdapter
```

EventBusClient implementation of the TransportPort interface.

Creates the underlying EventBusClient from a JSONP configuration file via `EventBusClient.from_config_sync()`.

34.4.1 Method: connect

Create the EventBusClient from the JSONP config and establish connection.

34.4.2 Method: disconnect

Close the EventBusClient connection.

34.4.3 Method: consume

Start consuming RPC requests for a service.

Subscribes to the routing key and blocks the calling thread. Incoming messages are adapted to the pika-style handler signature.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the service identifier.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for message binding.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match the configured exchange).
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

34.4.4 Method: rpc_call

Send an RPC request and wait for the response.

Creates a temporary subscription for the reply, sends the request with reply_to and correlation_id headers, and waits for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict.

34.4.5 Method: publish

Publish a message to the exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str, bytes, or dict /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties (used for headers).

Chapter 35

rabbitmq_adapter.py

35.1 Class: RabbitMQTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.rabbitmq_adapter import
↪  RabbitMQTransportAdapter
```

RabbitMQ implementation of the TransportPort interface.

35.1.1 Method: connect

Establish connection to RabbitMQ.

35.1.2 Method: disconnect

Close connection to RabbitMQ.

35.1.3 Method: connection

Access the underlying pika connection (for backward compat).

35.1.4 Method: stop_consuming

Signal the consume loop to exit.

35.1.5 Method: consume

Start consuming RPC requests for a service.

Sets up the exchange, queue, binding, and starts blocking consumption.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the queue name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for queue binding.

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

35.1.6 Method: rpc_call

Send an RPC request and wait for the response.
Creates a new connection for the RPC call (matching original behavior).
Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to serialize as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.
- `timeout`
  / *Condition*: optional / *Type*: int / *Default*: 30 /
  Timeout in seconds for waiting for the response.

Returns:

Parsed response dict.

35.1.7 Method: publish

Publish a message to an exchange.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body (str or bytes).
- `properties`
  / *Condition*: optional / *Type*: pika.BasicProperties /
  Optional pika.BasicProperties.

Chapter 36

fastapi_bridge.py

36.1 Class: FastAPIBridge

Imported by:

```
from MicroserviceBase.adapters.ui_bridge.fastapi_bridge import FastAPIBridge
```

FastAPI implementation of UIBridgePort.

Exposes a REST API that any UI client (Electron, browser, mobile) can call. Also provides a WebSocket endpoint for push-based service updates.

Endpoints: POST /api/request - Route a service request through the transport GET /api/services - Get current services info WS /ws/updates - Real-time service update stream

Requires `fastapi` and `uvicorn` (optional dependencies).

36.1.1 Method: start

Start the FastAPI server in a background thread.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable(request_data, exchange, routing_key) -> response dict.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable() -> services info dict.

36.1.2 Method: wait

Block the calling thread until the server thread exits.

Uses a loop with timeout so that KeyboardInterrupt can be caught on Windows.

36.1.3 Method: stop

Stop the FastAPI server.

36.1.4 Method: broadcast_update

Push a services update to all connected WebSocket clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

Chapter 37

exceptions.py

37.1 Class: ServiceError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceError
```

Base exception for service-related errors.

37.2 Class: TransportError

Imported by:

```
from MicroserviceBase.domain.exceptions import TransportError
```

Raised when a transport-level operation fails.

37.3 Class: ServiceNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceNotFoundError
```

Raised when a requested service is not found in the registry.

37.4 Class: MethodNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import MethodNotFoundError
```

Raised when a requested method is not found on a service.

Chapter 38

messages.py

38.1 Class: ResultType

Imported by:

```
from MicroserviceBase.domain.messages import ResultType
```

Result types for service responses.

38.2 Class: ServiceRequest

Imported by:

```
from MicroserviceBase.domain.messages import ServiceRequest
```

Structured request to a service method.

38.2.1 Method: to_dict

Converts this `ServiceRequest` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this request.

38.2.2 Method: to_json

Converts this `ServiceRequest` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this request.

38.2.3 Method: from_dict

Creates a `ServiceRequest` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the dictionary.

38.2.4 Method: from_json

Creates a `ServiceRequest` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the JSON string.

38.3 Class: ServiceResponse

Imported by:

```
from MicroserviceBase.domain.messages import ServiceResponse
```

Structured response from a service method.

38.3.1 Method: to_dict

Converts this `ServiceResponse` instance to an ordered dictionary.

Returns:

/ *Type*: OrderedDict /
Sorted ordered dictionary representation of this response.

38.3.2 Method: to_json

Converts this `ServiceResponse` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this response.

38.3.3 Method: get_json

Backward-compatible alias for `to_json()`.

Returns:

/ *Type*: str /
JSON string representation of this response.

38.3.4 Method: get_data

Backward-compatible alias for `result_data`.

Returns:

/ *Type*: Any /
The result data of this response.

38.3.5 Method: from_dict

Creates a `ServiceResponse` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the dictionary.

38.3.6 Method: from_json

Creates a `ServiceResponse` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the JSON string.

38.4 Class: ServiceEvent

Imported by:

```
from MicroserviceBase.domain.messages import ServiceEvent
```

Event emitted when service state changes.

38.4.1 Method: to_dict

Converts this `ServiceEvent` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this event.

38.4.2 Method: to_json

Converts this `ServiceEvent` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this event.

38.4.3 Method: from_dict

Creates a `ServiceEvent` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the dictionary.

38.4.4 Method: from_json

Creates a `ServiceEvent` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the JSON string.

Chapter 39

models.py

39.1 Class: MethodArgument

Imported by:

```
from MicroserviceBase.domain.models import MethodArgument
```

Describes a single argument of a service API method.

39.1.1 Method: to_dict

Converts this `MethodArgument` instance to a dictionary.
Returns:

/ *Type*: dict /
Dictionary representation of this argument.

39.1.2 Method: from_dict

Creates a `MethodArgument` instance from a dictionary.
Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing argument fields.

Returns:

/ *Type*: MethodArgument /
New instance populated from the dictionary.

39.2 Class: ServiceMethod

Imported by:

```
from MicroserviceBase.domain.models import ServiceMethod
```

Describes a single API method exposed by a service.

39.2.1 Method: to_dict

Converts this `ServiceMethod` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this method.

39.2.2 Method: from_dict

Creates a `ServiceMethod` instance from a name and dictionary.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  The method name.
- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing method fields.

Returns:

/ *Type*: ServiceMethod /
New instance populated from the dictionary.

39.3 Class: ServiceInfo

Imported by:

```
from MicroserviceBase.domain.models import ServiceInfo
```

Describes a service and its metadata.

39.3.1 Method: to_dict

Converts this `ServiceInfo` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this service info.

39.3.2 Method: from_dict

Creates a `ServiceInfo` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing service info fields.

Returns:

/ *Type*: ServiceInfo /
New instance populated from the dictionary.

Chapter 40

service_base.py

40.1 Class: ServiceBase

Imported by:

```
from MicroserviceBase.domain.service_base import ServiceBase
```

Pure domain base class for services.

All methods used to export APIs of a service must begin with the prefix 'svc_api'. This class handles API discovery, docstring parsing, and request dispatch. Transport and registry interactions are delegated to injected ports.

40.1.1 Method: get_service_info

Return the ServiceInfo dataclass for this service.

40.1.2 Method: get_service_info_dict

Return the service info as a plain dict (backward compat).

40.1.3 Method: serve

Start serving requests via the transport port.

40.1.4 Method: register_service

Register this service via the registry port.

40.1.5 Method: unregister_service

Unregister this service via the registry port.

40.1.6 Method: request_service

Send an RPC request to another service via the transport port.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys (backward compat format).

- `exchange_name`
  / *Condition*: required / *Type*: str /
  The exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  The routing key for the target service.

Returns:

/ *Type*: dict /
The response dict from the target service.

40.1.7 Method: close

Close the service.

40.1.8 Method: create_request_data

Create request data for a given method name and arguments.

40.1.9 Method: get_svc_api_methods_dict

Retrieve all service API methods (methods starting with 'svc_api').

40.1.10 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from their docstrings.

40.1.11 Method: parse_docstring

Parse function's docstring to get arguments and return information.

40.1.12 Method: svc_api_get_version

Get the service version.
Returns:

/ *Type*: str /
Version of the service.

40.1.13 Method: svc_api_get_gui_files

Compress and return GUI files as bytes.

40.1.14 Method: svc_api_get_gui_checksum

Get an MD5 checksum of all GUI files.
Returns None when gui_support is disabled or the GUIs directory is missing.
Returns:

/ *Type*: str /
MD5 hex-digest of the GUI files, or None.

40.1.15 Method: svc_api_shutdown

Gracefully shut down this service.

40.1.16 Method: is_specific_request

Check if the request is a specific request. Override in subclasses.

40.1.17 Method: on_specific_request

Handle a specific request. Override in subclasses.

Arguments:

- `request`
  / *Condition*: required / *Type*: str /
  The request method name.
- `body`
  / *Condition*: required / *Type*: dict /
  The parsed request body dict.

Returns:

/ *Type*: ServiceResponse /
A ServiceResponse, or None if not handled.

40.1.18 Method: dispatch_request

Dispatch a parsed request body and return a ServiceResponse.

This is the core domain logic: given a request dict with 'method' and 'args', find the matching API method, call it, and return a structured response.

Arguments:

- `request_body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse with result type and data.

40.1.19 Method: on_request

Handle an incoming request (transport callback signature).

This method is called by the transport adapter. It delegates to dispatch_request for the actual domain logic, then uses the transport to send the response back.

Arguments:

- `ch`
  / *Condition*: required / *Type*: object /
  Channel object from transport.

- `method`
  / *Condition*: required / *Type*: object /
  Delivery method from transport.
- `props`
  / *Condition*: required / *Type*: object /
  Message properties from transport.
- `body`
  / *Condition*: required / *Type*: bytes /
  Raw message body (bytes or dict).

Chapter 41

service_registry.py

41.1 Class: ServiceRegistry

Imported by:

```
from MicroserviceBase.domain.service_registry import ServiceRegistry
```

Pure domain registry that tracks services, handles aliases, and routes requests.

41.1.1 Method: handle_update

Handle a service registration/unregistration event.

Arguments:

- `service_information`
  / *Condition*: required / *Type*: dict /
  Dict with 'info' and 'state' keys.

41.1.2 Method: notify_updates

Notify connected clients about service updates via the registry port.

41.1.3 Method: svc_api_shutdown

Gracefully shut down the Service Registry. microservicebase-domain-service-registry-serviceregistry-svc-api-get-services-info ----------

Retrieve information of all services connected to the broker.

Returns:

/ *Type*: dict /
A dictionary containing information of all connected services.

41.1.4 Method: svc_api_get_realtime_update_exchange

Retrieve the exchange name of the realtime update exchange.

Returns:

/ *Type*: str /
The name of the realtime update exchange.

41.1.5 Method: svc_api_update_alias_conf

Update the alias configuration information.

Arguments:

- `alias_string`
  / *Condition*: required / *Type*: str /
  The alias configuration string to be updated.

Returns:

(*no returns*)

41.1.6 Method: svc_api_get_alias_conf

Retrieve the alias configuration string in JSON format.

Returns:

/ *Type*: str /
The alias configuration string in JSON format.

41.1.7 Method: is_specific_request

Check if the request is an alias-routed request.

41.1.8 Method: handle_alias_request

Handle an alias request by routing to the actual service.

Arguments:

- `body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse or raw response dict from the target service.

Chapter 42

factory.py

42.1 Function: create_transport

Create a TransportPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').
- `service_name`
  / *Condition*: optional / *Type*: str /
  Service name for help text (used by 'rabbitmq').
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: TransportPort /
A connected TransportPort instance.

42.2 Function: create_registry

Create a ServiceRegistryPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').

- `service_name`
  / *Condition*: optional / *Type*: str /
  Service name for help text (used by 'rabbitmq').
- `update_exchange_name`
  / *Condition*: optional / *Type*: str /
  Name for the realtime update fanout exchange.
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: ServiceRegistryPort /
A ServiceRegistryPort instance.

42.3 Function: create_ui_bridge

Create a UIBridgePort implementation.

Arguments:

- `bridge_type`
  / *Condition*: optional / *Type*: str /
  Bridge backend identifier. Currently supports 'fastapi'.
- `host`
  / *Condition*: optional / *Type*: str /
  Host to bind to.
- `port`
  / *Condition*: optional / *Type*: int /
  Port to bind to.

Returns:

/ *Type*: UIBridgePort /
A UIBridgePort instance (not yet started).

Chapter 43

registry.py

43.1 Class: ServiceRegistryPort

Imported by:

```
from MicroserviceBase.ports.registry import ServiceRegistryPort
```

Abstract interface for service registration and discovery.
Implementations handle how services announce themselves and how the registry discovers/tracks them.

43.1.1 Method: register

Register a service with the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

43.1.2 Method: unregister

Unregister a service from the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

43.1.3 Method: subscribe_to_events

Subscribe to service registration/unregistration events.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body) for events.

43.1.4 Method: notify_update

Broadcast a services update to all subscribers.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

43.1.5 Method: get_update_channel_name

Return the name of the realtime update channel/exchange.

Returns:

Channel/exchange name as a string.

Chapter 44

transport.py

44.1 Class: TransportPort

Imported by:

```
from MicroserviceBase.ports.transport import TransportPort
```

Abstract interface for message transport.

Implementations provide the actual connectivity (e.g., RabbitMQ, Redis, MQTT). The domain layer depends only on this interface.

44.1.1 Method: connect

Establish connection to the message broker.

44.1.2 Method: disconnect

Close connection to the message broker.

44.1.3 Method: stop_consuming

Signal the consume loop to stop.

44.1.4 Method: consume

Start consuming messages for a service.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name of the service (used as queue name).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key to bind the queue.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body) for incoming messages.

44.1.5 Method: rpc_call

Send an RPC request and wait for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict to serialize and send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict from the target service.

44.1.6 Method: publish

Publish a message to an exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties dict.

Chapter 45

ui_bridge.py

45.1 Class: UIBridgePort

Imported by:

```
from MicroserviceBase.ports.ui_bridge import UIBridgePort
```

Abstract interface for UI communication bridge.

Implementations provide the actual UI connectivity (e.g., FastAPI REST, gRPC). Any frontend (Electron, browser, mobile) can connect through this port.

45.1.1 Method: start

Start the UI bridge server.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable that accepts a ServiceRequest dict and returns a ServiceResponse dict. Used to route UI requests to services.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable that returns the current services info dict.

45.1.2 Method: stop

Stop the UI bridge server.

45.1.3 Method: broadcast_update

Push a services update to all connected UI clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

Chapter 46

Appendix

About this package:

Table 46.1: Package setup

Setup parameter	Value
Name	MicroserviceBase
Version	0.1.1
Date	02.02.2023
Description	Microservice base framework with pluggable transport and registry adapters
Package URL	python-microservice-base
Author	Nguyen Huynh Tri Cuong
Email	Cuong.NguyenHuynhTri@vn.bosch.com
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.10
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 47

History

0.1.0	02/2023
Initial version	

MicroserviceBase.pdf
Created at 09.02.2026 - 14:12:31
by GenPackageDoc v. 0.16.0